



LeetCode 1945: Sum of Digits of String After Convert

1. Problem Title & Link

Title: LeetCode 1945: Sum of Digits of String After Convert

Link: <https://leetcode.com/problems/sum-of-digits-of-string-after-convert/>

2. Problem Statement (Short Summary)

Given:

- A string s containing lowercase English letters
- An integer k

Perform the following operations:

Step 1: Convert Letters to Numbers

$a \rightarrow 1$

$b \rightarrow 2$

$c \rightarrow 3$

...

$z \rightarrow 26$

Concatenate all numbers into a string.

Step 2: Transform k Times

Replace the number with the sum of its digits.

Repeat exactly k times.

Return the final result.

Example

Input:

$s = \text{"iiii"}$

$k = 1$

Convert:

$i = 9$

9999

Digit Sum:

$9 + 9 + 9 + 9 = 36$

Output:

36



3. Examples (Input → Output)

Example 1

Input:

`s = "iiii"`

`k = 1`

Output:

36

Example 2

Input:

`s = "leetcode"`

`k = 2`

Convert:

`l = 12`

`e = 5`

`e = 5`

`t = 20`

`c = 3`

`o = 15`

`d = 4`

`e = 5`

12552031545

1st Transformation:

$1+2+5+5+2+0+3+1+5+4+5$

$= 33$

2nd Transformation:

$3+3$

$= 6$

Output:

6

Example 3



Input:

$s = \text{"zbax"}$

$k = 2$

Convert:

26 2 1 24

262124

1st Transform:

$2+6+2+1+2+4$

$= 17$

2nd Transform:

$1+7$

$= 8$

Output:

8

4. Constraints

$1 \leq s.length \leq 100$

$1 \leq k \leq 10$

Important:

Letters are lowercase English letters only.

5. Core Concept (Pattern / Topic)

String Simulation

Secondary Topics:

- Character Conversion
- Digit Sum
- Mathematical Simulation

6. Thought Process (Step-by-Step Explanation)

Step 1: Convert Characters

For every character:

value = position in alphabet



Formula:

$$\text{ord}(\text{ch}) - \text{ord}('a') + 1$$

Example:

c

$$= 99 - 97 + 1$$
$$= 3$$

Example

Input:

abc

Convert:

a $\rightarrow$ 1

b $\rightarrow$ 2

c $\rightarrow$ 3

Result:

123

Step 2: Repeated Digit Sum

Example:

123

Transform:

$$1 + 2 + 3$$
$$= 6$$

If:

k = 2

Transform again:

6

Result remains:

6

Key Observation

This is not DP.

This is not Greedy.



This is a pure:

Simulation Problem

Just follow the steps exactly.

7. Visual / Intuition Diagram (ASCII Diagram)

Example:

`s = "zbax"`

`k = 2`

Convert:

`z b a x`

`26 2 1 24`

Join:

`262124`

Transform 1:

`2+6+2+1+2+4`

`= 17`

Transform 2:

`1+7`

`= 8`

Answer:

`8`

8. Pseudocode (Language Independent)

```
numString = ""

for each character
    convert to alphabet position
    append to numString

number = numString

repeat k times

    sumDigits = 0

    for each digit

        sumDigits += digit
```



```
        number = sumDigits

    return number
```

9. Code Implementation

Python

```
class Solution:
    def get Lucky(self, s: str, k: int) -> int:

        num_string = ""

        for ch in s:
            num_string += str(ord(ch) - ord('a') + 1)

        current = num_string

        for _ in range(k):

            digit_sum = 0

            for digit in current:
                digit_sum += int(digit)

            current = str(digit_sum)

        return int(current)
```

Optimized Python

```
class Solution:
    def get Lucky(self, s: str, k: int) -> int:

        total = 0

        for ch in s:

            value = ord(ch) - ord('a') + 1

            for digit in str(value):
                total += int(digit)

        k -= 1

        while k > 0:

            next_sum = 0

            while total > 0:
```



```
        next_sum += total % 10
        total //= 10

    total = next_sum
    k -= 1

    return total
```

Java

```
class Solution {

    public int getLucky(String s, int k) {

        StringBuilder sb =
            new StringBuilder();

        for(char ch : s.toCharArray()) {

            sb.append(ch - 'a' + 1);
        }

        String current = sb.toString();

        for(int i = 0; i < k; i++) {

            int sum = 0;

            for(char digit :
                current.toCharArray()) {

                sum += digit - '0';
            }

            current = String.valueOf(sum);
        }

        return Integer.parseInt(current);
    }
}
```

10. Time & Space Complexity

Let:

n = length of string

Time Complexity



$O(n + k \times \text{digits})$

Since:

$n \leq 100$

$k \leq 10$

Very small.

Space Complexity

$O(n)$

For converted string.

11. Common Mistakes / Edge Cases

Mistake 1

Using:

`ord(ch)`

directly.

Wrong:

`a = 97`

Need:

`ord(ch) - ord('a') + 1`

Mistake 2

Forgetting that:

`z = 26`

produces two digits.

Need:

`2 + 6`

during transformation.

Mistake 3

Performing:

`k-1`

transformations accidentally.

Must perform exactly:

`k`

times.



Edge Cases

Single Character

Input:

a

k = 1

Convert:

1

Output:

1

Largest Character

Input:

z

k = 1

Convert:

26

Output:

2+6=8

12. Detailed Dry Run

Input:

s = "leetcode"

k = 2

Convert:

12 5 5 20 3 15 4 5

Join:

12552031545

Transformation 1:

Digit	Running Sum
1	1
2	3
5	8



5	13
2	15
0	15
3	18
1	19
5	24
4	28
5	33

Result:

33

Transformation 2:

$3 + 3$

$= 6$

Return:

6

13. Common Use Cases (Real-Life / Interview)

Data Transformation

Convert symbols into numeric representations.

Encoding Systems

Letter-to-number mapping.

Digital Root Style Problems

Repeated digit reduction.

Interview Purpose

Tests:

Simulation

String Processing

Character Manipulation

14. Common Traps (Important!)

Trap 1

Treating:



26

as:

26

instead of:

2 + 6

during digit sum.

Trap 2

Confusing alphabet position with ASCII value.

Trap 3

Stopping after first transformation when:

$k > 1$

15. Builds To (Related LeetCode Problems)

LC 1945 - Sum of Digits After Convert

↓

LC 258 - Add Digits

↓

LC 2243 - Calculate Digit Sum of a String

↓

LC 415 - Add Strings

↓

LC 67 - Add Binary

These strengthen simulation and digit-manipulation skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Direct Simulation	$O(n)$	$O(n)$	Best
Optimized Digit Sum	$O(n)$	$O(1)$	Advanced

Approach 1: Direct Simulation (Chosen)

Build converted string.

Apply transformations.

Easy to understand.



Approach 2: Optimized

Avoid creating the full converted string.

Compute digit sums directly.

Slightly better memory usage.

17. Why This Solution Works (Short Intuition)

The problem explicitly defines a sequence of operations:

1. Convert letters to alphabet positions.
2. Sum the digits.
3. Repeat k times.

By simulating these steps exactly, we guarantee the correct answer.

18. Variations / Follow-Up Questions

Follow-Up 1

Perform transformations until a single digit remains.

Related:

Conceptually similar to **Add Digits (LC 258)**.

Follow-Up 2

Support uppercase letters.

How would conversion change?

Follow-Up 3

Use custom mappings.

Example:

A → 100

B → 200

Follow-Up 4

Convert digits back to characters after transformation.

Interview Takeaway

This is a classic **Simulation Problem**.

The key skill is:

Read carefully

Follow instructions exactly



Implement step-by-step

Whenever you hear:

Perform operation repeatedly

Transform data multiple times

Follow given process

think:

Simulation

Many easy-to-medium interview questions are solved simply by accurately simulating the described process.